PROBLEMS OF IMPLEMENTING MUSICAL COUNTERPOINT GENERATION SYSTEMS THROUGH THE PROLOG COMPUTER LANGUAGE

Robert Wolf
Elder Conservatorium of Music
The University of Adelaide

ABSTRACT

This paper discusses typical problems associated with the construction of systems for the automatic generation of musical counterpoint, and how these problems can be overcome by using the PROLOG computer language and its unique features.

1. INTRODUCTION

The rules of counterpoint grow organically from the laws of psychoacoustics. For example, in the first species counterpoint only perfect and imperfect consonances can be used. In acoustical terms it means that the relations between notes can be expressed by *rational numbers* with small values used for both *numerator* and *denominator*, such as 2/1 (perfect octave), 3/2 (perfect fifth), 4/3 (perfect fourth, which is allowed in 3 or more voices counterpoint), 5/4 (major third) and 6/5 (minor third). Of course, on equally tempered instruments these values are only close approximations. Nevertheless, the fact remains that contrapuntal rules are in direct relation with mathematical proportions between frequencies. It is almost guaranteed that a composition based on these rules will sound natural to a listener who grew up in any of the western countries.

Today, in the era of computers, we tend to seek alternative methods for organising musical material, such as Markov chains, chaos theory, fractals, and many others, mostly based on mathematical models of what we perceive as natural. Such methods are quite natural for computers, which have no connection with the nature of sound. However, it is always a challenge and a good exercise to apply computational power to compose counterpoints.

On the surface, the construction of a system for the automatic generation of counterpoint seems to be quite a trivial task. Contrapuntal rules present themselves as easily programmable. However, after closer examination, several difficult problems arise. These problems are presented here and a method of overcoming them by using the PROLOG programming language is proposed.

This paper proposes using the PROLOG computer language to construct a contrapuntal generation system. The declarative nature of this language, and several another unique features, make it a suitable tool for constructing an expert system specialising in the rules of counterpoint. A system of this kind has been constructed at the University of Adelaide.

2. HISTORIC BACKGROUND

Counterpoint generation systems have already been successfully constructed before. The first such system was written by William Schottstaedt from Stanford University (Schottstaedt 1984). He used the SAIL programming language (*Stanford Artificial Intelligence Language*). Kemal Ebcioglu constructed his CHORAL system for automatic harmonisation of four-part chorales (Ebcioglu 1988). He acknowledged the potential usefulness of PROLOG. However, at the time, PROLOG implementation were simply too slow and not performing well enough. Therefore, he devised his own PROLOG-like language called BSL (*Backtracking Specification Language*). Ebcioglu's CHORAL system is based on roughly 350 rules and a backtracking mechanism. An interesting solution was proposed by David Cope. His GRADUS program, written in LISP, is based on a self-learning principle. His system learns from its own mistakes by maintaining a database of melodic dead ends (Cope 2005). The implementation of a counterpoint generation system as a logic constraint satisfaction problem was proposed by Ovans (Ovans 1990). Anders also recognises the usefulness of PROLOG in this context (Anders 2003).

3. PROLOG LANGUAGE

The PROLOG computer language was created in Marseille around 1972 by a group of researchers led by Alain Colmerauer. Initially it was intended to be just a tool for construction of a software system able to understand the French language in its written form (Colmerauer *et al.* 1992). The name PROLOG stands for PRO-gramming in LOG-ic.

Over the years several misconceptions about PROLOG have arisen. It is frequently perceived as some form of esoteric or ancient dialect, totally outdated with dubious usability. In reality, it is a general purpose language. It possesses several unique features, which can not be found in other mainstream languages available today (Bratko 2000).

It is perhaps necessary to list and briefly to describe several unique features of PROLOG, which are beneficial in the context of implementing a counterpoint generating system.

3.1. Declarative style

In general, a computer program consists of a series of instructions, which lead to solving a particular problem. A program written in PROLOG differs substantially from this model. In PROLOG, a program describes *what* has to be solved instead of *how* to solve it. Instead of writing a sequence of instructions on *what to do,* a programmer specifies the problem itself. The machine *reasons* upon such specification, and in consequence is able to devise the algorithm automatically. This is called Declarative Style, as opposed to Imperative Style; the later is predominant in computer programming today.

3.2. Symbolic calculations

Typically, computers work on numbers. PROLOG is one of the few languages able to process symbols. This feature frees the programmer from encoding everything into numerals, and makes computer code more human friendly.

3.3. In-built Inference Mechanism

Inference is the ability to make deductions, and to find solutions based on the facts and rules provided. In other words, the language itself has the ability to find ways of solving particular problems without the programmer telling the machine precisely how to do it. From a theoretical point of view, PROLOG's deductive system is based on so called *first order logic*. It is a formal method of processing logical implications (Bratko 2000).

3.4. Backtracking

Backtracking is one of the parts of the in-built inference mechanism. It is the ability to reverse the order of execution. In particular, when the program reaches a dead end (no solution), it retraces its steps, trying to find alternative solutions.

3.5. Meta-logic ability

This is the ability to write programs which operate on other programs. It is similar to lambda expressions, which can be found in LISP and several other languages. However, in PROLOG it is even possible to implement primitive control structures, such as IF..THEN..ELSE conditions, WHILE..DO loops and many others. It is also possible to modify the way in which PROLOG programs work, by defining alternative methods of processing facts and rules. It is even possible to implement different inference mechanisms.

All the aforementioned features make PROLOG a very powerful tool, especially for implementing expert systems. In practice, however, PROLOG requires some experience from the programmer to be able to unleash its entire potential. Nevertheless, with skilful programming, it is possible to push a lot of thinking activity onto the machine.

4. COMMON PROBLEMS

During the construction of a contrapuntal expert system, several problems have been identified, analysed and compared with examples available in the literature. They are all presented below, in brief, and effective solutions in PROLOG are proposed.

4.1. Declarative Nature of the Rules of Counterpoint

The rules of counterpoint are not written in a way which guarantees plausible outcomes. In particular, they do not provide any series of steps which could be translated into an algorithm. They do not specify whether the composer should start composing the melody from the beginning, from the end, or from the middle. Instead, they tell us what is good and what is bad. They are declarative in their nature. They do not tell us how to compose. It is entirely up to the composer how he or she comes up with melodies. This situation presents the quintessential problem to the computer programmer. Among all the contrapuntal rules there is only little written about how to start composing a composition.

Computer languages are mostly imperative in their nature. A computer program is usually nothing more than a series of instructions on how to achieve a solution step by step. This paradigm gap forces the computer programmer actually to devise a system for making compositions.

This particular approach takes advantage of the declarative nature of PROLOG, which is similar to the declarative nature of the contrapuntal rules. In particular, the rules can be translated from English to PROLOG without specifying any instructions on how to compose. PROLOG's internal inference mechanism takes care of generating melodies by itself.

4.2. Melodic Dead Ends

Typically, a composer makes a series of choices when writing a counterpoint. Sometimes the melody comes to a dead end, where contrapuntal rules prohibit any further continuation. In such cases the eraser is used; several previous choices are invalidated and alternative notes are chosen. Unfortunately, computer programs can only keep moving forward or crash. The majority of modern computer languages carry an in-built mechanism for exception handling. Nevertheless, the computer programmer needs to find a way of keeping track of already tried dead end solutions, cancelling part(s) of the current solution, and making alternative choices.

PROLOG's unique backtracking ability makes it particularly effective for dealing with melodic dead ends.

4.3. Music Notation versus Data Representation

To calculate counterpoint, the computer must be able to work on musical notes. In other words, musical knowledge somehow has to be represented inside the machine. A good contrapuntal generation system will also make distinctions between notes and intervals from their enharmonic equivalents. Within this context it is beneficial to use symbols and symbolic calculations instead of encoding everything into numbers.

The musical information that has to be encoded formally would include the following: notes with alterations and their respective octaves; pauses; rhythmic values; intervals (with both chromatic and diatonic parts included); melodic lines; and multi-staved compositions. In an object-orientated programming language, dedicated data types shall be defined for each of those entities. Additionally, a number of sub-routines (or methods) for accessing data within the structures would need to be constructed (Cope 2005) (Anders 2003).

In PROLOG, data structures do not need to be defined explicitly before use. It is enough to focus on preparing accessing methods only. Additionally, PROLOG can perform symbolic calculations. Therefore, information such as note names and alterations can be expressed directly as symbols, without encoding them as numbers.

Another interesting feature of PROLOG is the fact that methods designed for performing one particular transformation are frequently able to do the inverse transformation as well. A typical scenario would be to create a program to calculate musical intervals between two different notes. The reverse function of such program is to calculate notes based on the interval provided. In the majority of languages, this would require to create two different blocks of code, one for calculating intervals, another for calculating notes. In PROLOG, however, it is usually possible to use exactly the same code to do both.

4.4. The Contrapuntal Rule-Set

In a typical scenario, the contrapuntal rule-set is a moving target. The computer programmer faces a difficult question: “should I use J.J. Fux (Fux 1725/1965)? Or should I select Knud Jeppesen (Jeppesen 1939)? Should I do it in modal scales or major/minor?”. In practice, this can lead to a situation where a computer program specialises in one particular contrapuntal style. Therefore, it is necessary to design its architecture so that contrapuntal rule sets can be exchangeable. In particular, the set of rules responsible for keeping the counterpoint in a particular musical style should be designed as a replaceable module. This way, one and the same program can be used for generating both modal and tonal counterpoints. In such architecture, the only thing that has to be changed is a reference to a particular scale-assessing module.

In this respect, PROLOG, with its meta-logic capacity can be very useful.

4.5. The Choice

The process of finding counterpoint for a particular cantus firmus can be considered as finding a solution to a problem. Composing a counterpoint resembles to an extent solving a puzzle, such as Sudoku. The main difference is that Sudoku always has one possible solution, while there are multiple solutions for any given cantus firmus. As long as the composer strictly adheres to one particular set of contrapuntal rules, the composition process can be treated as choosing one possible solution which seems most pleasing to the composer. Typically, the number of first species counterpoints for a 12-note-long melody can reach up to several thousands. However, the number of second and third species counterpoints is so high that it can easily exceed million.

In this context the counterpoint generating program is nothing more than a mechanism for finding solutions to a set of logical constraints (Anders 2003). PROLOG with its inference engine based on first order logic is very well suited for this purpose. In particular, it is not necessary to code the entire searching mechanism. It is sufficient only to translate the rules of counterpoint into PROLOG's clauses. All possible counterpoints are calculated automatically by language's in-built mechanisms through successful application of the encoded rules.

The solutions found by PROLOG are not generated in random order. Rules are applied in the order in which they are written in the code. Therefore, the first solution is always the same, and reflects the ordering of the rules. In cases where only one, randomly selected solution is required, PROLOG's inference mechanism can be easily instructed to modify its own working and to apply rules randomly.

4.6. Aesthetics

The first counterpoint found may not necessarily be the most pleasing one. The rules of counterpoint offer several guidelines on what is better and what should be avoided. For example, they state that the composition should contain more imperfect consonances than perfect ones. These kinds of contrapuntal guidelines can be easily translated into a set of evaluation rules. These rules can be used to assess why some counterpoints are better than others. In this context, an automatic counterpoint generating system can be implemented as a search mechanism. It should apply contrapuntal rules in random order and keep looking for alternative solutions until it finds the one with a higher score than some predetermined threshold.

5. A CONTRAPUNTAL SYSTEM IN PROLOG

Typically a program written in PROLOG displays properties of either expert or knowledge based systems. In such systems, the role of a programmer is to encode a domain knowledge in formal notation. It could be said that the computer programmer assumes the role of a knowledge engineer. One of the unique features of

PROLOG is its ability to deduct automatically a solution based on this knowledge. In other words, PROLOG comes with an inbuilt inference mechanism, which can be considered as a form of machine intelligence, making the task of the programmer easier.

This particular contrapuntal system was constructed in a bottom-up fashion, starting from modules responsible for exporting calculated counterpoints to score documents and ending in the top-level contrapuntal solvers. The construction of the system is described briefly below.

The dialect of PROLOG used for the construction was HERCs Prolog, written by the author. It is loosely based on Micro-PROLOG dialect, with several MIDI and music related features.

5.1. Data Representation

The construction of the system was started by making several decisions on how musical data are represented and processed. The entire composition is denoted as a list of melodies. Each melody is a list of notes. Each note is a pair consisting of pitch and a number specifying duration. Each pitch is a pair consisting of a symbol specifying note and a numerical octave value. The symbols for notes are: C, C#, Cx, Cb, Cbb, D, D#, Dx, Db, Dbb... and so on. An empty pair ([]) denotes a pause. For the durations the following numbers were used: 12 – eight, 24 – crotchet, 48 – minim, 96 – longa. Hence, the following musical example



Figure 1. Sample melody.

can be formally notated as:

[[[C 1] 48] [[G 1] 48] [[A 1] 24] [[Bb 1] 24] [[A 1] 24] [[G 1] 24] [[F# 1] 24] [[G 1] 24] [[D 2] 24] [[] 24]]

A cantus firmus can also be represented with durations omitted. Hence the following cantus firmus:

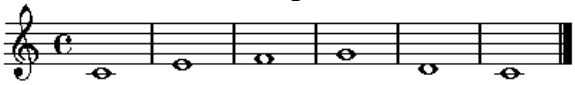


Figure 2. Sample cantus firmus.

can be denoted as:

[[C 1] [E 1] [F 1] [G 1] [D 1] [C 1]]

5.2. Input and Output of the System

Cantus firmus can be entered into the system directly from the terminal using the aforementioned notation. It can also be entered using external MIDI keyboard. The incoming MIDI transmission becomes translated directly into the internal PROLOG representation.

The calculated counterpoints, together with the original cantus firmus, are exported into PDF documents using Lilypond music engraving software. Large number of counterpoints can be exported in bulk into text files if score conversion is not required.

5.3. Interval Calculator

The next constructed module was the interval calculator. It is capable of calculating intervals between two provided notes. Each interval consists of a pair of numbers, where the first number is a diatonic interval, while the second is chromatic. For example, the interval between C and C# is an augmented prime, and can be denoted as [0 1]. It is different from the minor second (as between C and D flat), denoted as [1 1]. Similarly, the distinction between augmented fourth and diminished fifth is evident in the code. The former is denoted as [3 6] while the later as [4 6].

The definition of interval calculator (INTERVAL) works in a fashion that is typical for PROLOG. In particular, it can be used for calculating intervals between notes and calculating notes from the interval provided. For example:

[INTERVAL [C 1] [G 1] *x] will calculate *x = [4 7]
[INTERVAL *x [G 1] [4 7]] will calculate *x = [C 1]
[INTERVAL [C 1] *x [4 7]] will calculate *x = [G 1]

There is also a more generic version, which can calculate parts of notes and parts of intervals. For example:

[interval [C 1] *x [*i 7]] will calculate all three possible combination of values for *x and *i:

*x = [G 1], *i = 4, which is perfect fifth
*x = [Fx 1], *i = 3, which is double augmented fourth
*x = [Abb 1], *i = 5, which is double diminished sixth

5.4. Translation of the Contrapuntal Rules

The bulk of the system code consists of the rules of counterpoint translated directly into PROLOG clauses. They can be divided into the following classes.

5.4.1. Facts

Facts are unconditional truth. They are written as clauses without conditions. In other words, they are statements, which are always true. For example, a step interval (major or minor second up or down) can be coded as the example below. In a similar way, skip and leap intervals can be entered into the system.

[[step_interval [1 2]]]
[[step_interval [1 1]]]
[[step_interval [-1 -2]]]
[[step_interval [-1 -1]]]
[[skip_interval [2 4]]]
…. and so on.

5.4.2. Rules

Rules describe sentences which are true under certain conditions. For example, we may try to say that a melody can only go by step (up or down) or a skip (either direction). One possible way to express this in PROLOG is to use two rules, one saying that melodic interval can be a step, while the other would say it can be a skip.

[[melodic_interval *i] [step_interval *i]]

```
[[melodic_interval *i] [skip_interval *i]]
```

5.4.3. Prohibited constructions

Typically they have several checks, point of no return and a forced failure. A good example of such a rule is prohibited perfect fifths in similar motion. Two rules are necessary. One for prohibited perfect fifths, another for all other cases.

```
[[prohibited_parallel_intervals *i *i] [perfect_interval *i] / fail]

[[prohibited_parallel_intervals *i1 *i2]]
```

The first rule detects the occurrence of two identical perfect intervals. Then, the "cut" operator indicates the point of no return, which is immediately followed by failure. The second definition is used for all other cases.

5.5. Species Selector

The counterpoint generation system should be able to produce all species. As the fifth species counterpoint is a combination of all other species, the system should also be able to generate sequences of bars in different species. However, species can not be mixed completely freely. There are some rules to it as well. In particular, the last bar should always be of the first species, and a fourth species bar cannot be followed by a first species bar.

The construction of a species selector module can be started by translating the rules from English to PROLOG. Initially, we will choose some symbols for bars in different species, such as: bar1, bar2, bar3 and bar4 for the first, second, third and fourth species, respectively. Now we need to tell the PROLOG system that those symbols represent species. It can be expressed as follows:

```
[[species bar1]]
[[species bar2]]
[[species bar3]]
[[species bar4]]
```

Now we will say that the last bar can be of the first species only:

```
[[last_bar bar1]]
```

The next rule will say that the initial bar can be of any species. The rules uses a variable (beginning with star "*"), which has to be a symbol used for a species.

```
[[first_bar *B] [species *B]]
```

Our next step is to define how species can be chained in a sequence.

```
[[species_sequence bar1 *B] [species *B]]
```

This means that a first species bar can be followed by any other species bar.

```
[[species_sequence bar2 *B] [species *B]]
[[species_sequence bar3 *B] [species *B]]
```

This means the same thing for second and third species.

```
[[species_sequence bar4 bar2]]
[[species_sequence bar4 bar3]]
[[species_sequence bar4 bar4]]
```

These three rules mean that a fourth species bar can only be followed by second, third and fourth species.

Now, we can put all this together and create the species selector module. It will take a cantus firmus as the input and create a sequence of bars in different species with the same length as the cantus firmus. It consists of three PROLOG rules. It should be noted that the entire cantus firmus is denoted as *cf. It can be also described as [*cf1 *cf2 : *cfs], where *cf1 is the first note, *cf2 is the second and whatever follows the colon mark is the list consisting of the reminder of the notes. Similar notation is used for bars: such as [*bar1 : *bars], etc.

The first rule says that the first bar must obey the rules regarding the initial bar. When this restriction is satisfied, the species selector can make sure that the rest conform with the rules by calling species_selector_cd rule (which is described later).

```
[[species_selector *cf [*bar1 :*bars]]
        [first_bar *bar1]
        [species_selector_cd *cf [*bar1 : *bars]]
]
```

The second rule deals with the last bar. It should be noted that colon marks are not present, indicating that there are no more remaining notes.

```
[[species_selector_cd [*cf1 *cf2] [*bar1 *bar2]]
        [last_bar *bar2]
]
```

Our last rule takes care of all the notes between the first and the last one. It should be noted how this rule calls itself recursively, and how it strips the notes and bars from the head of the lists. For example, [*cf1 : *cfs] is passed to the recursive call without *cf1, i.e. as *cfs only. The same treatment is applied to the list of bars, i.e. [*bar1 *bar2 : *bars] becomes [*bar2 : *bars].

```
[[species_selector_cd [*cf1 :*cfs] [*bar1 *bar2 :*bars]]
        [species_sequence *bar1 *bar2]
        [species_selector_cd *cfs [*bar2 : *bars]]
]
```

Now the species selector can be applied to the sample cantus firmus, by formulating a PROLOG query:

```
[res
    [species_selector [[C 1] [E 1] [D 1] [C 1]] *bars]
    [show *bars]
]
```

The result will be, of course: [bar1 bar1 bar1 bar1]. All possible combinations can be generated by inducing unconditional failure at the end of the query. In this case, PROLOG will try all available solutions.

```
[res
    [species_selector [[C 1] [E 1] [D 1] [C 1]] *bars]
    [show *bars]
    fail
]
```

These rules illustrate that the species selector can be written in PROLOG and the thinking process behind its construction. Of course, such a module should be expanded to accommodate some extra code for generating skeletal rhythmic patterns for counterpoint. It should be noted that the rules will be applied in the same order as they were written. Therefore it might be useful to modify it slightly to make it possible to apply the rules in random order.

5.6. Scale Selector

A well designed counterpoint generation system should not be specific to any particular scale or mode. It should also be able to use pitch alterations appropriately within the enharmonic context. For this purpose, a mechanism for describing musical scales needs to be implemented.

It has to be noted that a musical scale or mode is not just a collection of pitches. It also carries some rules on how melodies can be constructed. For example, in minor scales, the leading seventh tone is raised whenever melody goes up, etc. (Polglase 2008).

Each musical scale can be constructed using exactly the same methodology as the one described previously about the species selector module. In particular, a scale evaluating module will contain rules on how notes can be combined into melodic sequences. Such a module can function as a kind of plug-in component to the rest of the system. In other words, the counterpoint generating system can be written without worrying about any scale-related intricacies. This functionality can be provided by external modules, specifically designed for all different musical scales.

5.7. Species Solver

Each species of counterpoint will need its own dedicated set of rules. For example, the first species counterpoint needs to worry about consonance, parallel perfect intervals and several others. The second species counterpoint introduces rules regarding the use of dissonances. The third species introduces even more devices, among them nota cambiata. And so on.

The fact that the florid counterpoint is a collection of bars in different species, complicates the architecture of the system. It is therefore practical to design 4 different species solvers, which can deal with just a single bar. The duty of applying correct rules to the entire melodies is now pushed to another module.

5.8. Counterpoint Solver

The duty of the counterpoint solver is just to apply different species solvers to each bar of the counterpoint. It works recursively in a loop in similar way to the species selector. Aside from counterpoints, this solver also returns some more information, such as: species structure, list of vertical intervals (against cantus firmus) and list of horizontal intervals within the melody.

5.9. Main Program

The main program can be constructed by putting all aforementioned modules together. In particular, it can work in the following sequence:

1. Use MIDI keyboard to input cantus firmus.

2. Use species selector module to generate rhythmic pattern of the entire counterpoint.

3. Use counterpoint solver with the cantus firmus and a specific scale (i.e. A-minor) to produce counterpoint.

4. Export generated score and possibly play it on a MIDI enabled instrument.

A surprising functionality of such a system may occur. In practice, PROLOG programs display the ability to perform the reverse transformation. In this case, the counterpoint generating system can, in fact, reconstruct all possible canti firmi if a counterpoint was provided as the input. It also has to be noted that species sequences can be predetermined or generated by the system. In this respect, it is possible to limit the system to generating only one particular species of counterpoint, or florid counterpoints with rhythmic structures provided by the user.

6. CONCLUSIONS

The main benefit of using PROLOG programming language for construction of the counterpoint generating system is the fact that the programmer is not required to design and implement any traditional algorithms. It is sufficient simply to translate contrapuntal rules from English to PROLOG. The language is able to find counterpoints entirely by itself when trying to satisfy logical constraints described as rules. Also, the rules of counterpoint are not written in algorithmic fashion. They do not specify how to construct a counterpoint melody. Instead, they describe what is good and what is prohibited, without giving any recipe. The declarative programming style of PROLOG overcomes this problem, thus making it the ideal tool for building this type of system.

7. REFERENCES

[1] Anders, T, *Composing Music by Composing Rules: Computer Aided Composition employing Constraint Logic Programming*. Sonic Arts Research Centre, Queens University Belfast, Northern Ireland, 2003.

[2] Bratko, I. *Prolog Programming for Artificial Intelligence*. Reading, MA: Addison Wesley, 2000.

[3] Colmerauer, A. & Roussel, P. *The Birth of Prolog*. <http://alain.colmerauer.free.fr/ArchivesPublications/HistoireProlog/19November92.pdf>, 2009.

[4] Cope, D. *Computer Models of Musical Creativity*. Cambridge, MA: MIT Press, 2005.

[5] Ebcioglu, K. "An Expert System for Harmonizing Four-Part Chorales" in the Style of J. S. Bach" in *Journal for Logic Programming* 8 (), 1990, pp. 145-185.

[6] Fux, J. J. *The Study of Counterpoint*. New York: W. W. Norton & Company, 1965,

[7] Jeppesen, K. *Counterpoint: The Polyphonic Vocal Style of the Sixteen Century*. New York: Prentice-Hall, Inc., 1939.

[8] Ovans, R. "Music Composition as a Constraint Satisfaction Problem", in *Proceedings of the International Computer Music Conference*, Glasgow, UK, 1990, pp 317-319.

[9] Polglase, J. *Contrapuntal Analysis & Composition 2008 Semester Two. Course Outline*. Adelaide: University of Adelaide, 2008.

[10] Schottstaedt, W. "Automatic Species Counterpoint". Stanford Technical Report Stan-M-19. Stanford: Department of Music, Stanford University 1984.